

1

2

3

NOCK & THE ART OF
SOLID-STATE COMPUTING
An Enquiry into the Virtual

4

N. E. Davis
~lagrev-nocfep

5

6 Contents

7	1 The Roots of Computing in the Ancient World	1
8	1.1 Tabulation	3
9	1.2 Solid-state indices	3
10	2 Combinators for Fun and Profit	5
11	2.1 Combinators in Computing	5
12	2.1.1 The SKI calculus	6
13	2.1.2 The BCKW calculus	8
14	2.2 Nock Opcodes as Combinator	8
15	2.2.1 I Identity = Opcode 0	10
16	2.2.2 K Constant = Opcode 1	10
17	2.2.3 S Substitution = Opcode 2	10
18	2.2.4 The Axiomatic Operators	10
19	2.3 Idioms & Design Patterns	10
20	2.3.1 C Reversal	11
21	2.4 Virtualization	11
22	3 Interpreter Runtime	13

Chapter 1

The Roots of Computing in the Ancient World

Abstract

Cuneiform and early computational concepts are explored in this chapter.

Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature.

(Keiji Nishitani, *Religion and Nothingness*, 1982, pp. 129–130)

There are many worthy raconteurs of the history of computing, such as **Hofstadter1980** (**Hofstadter1980**) and **Penrose2004**

42 (Penrose2004). What distinguishes my approach to the his-
43 tory of computing from other tellings? Computing has deep
44 connexions to alchemy and true language, which I hope to
45 demonstrate without assuming any Hermetic background on
46 the reader's part. Much like technology, computing is not merely
47 a tool but a stance towards the world. However, it does not en-
48 tail technologism's supercession; computing is always both a
49 lens and a kaleidoscope, through which many layers of reality
50 can be seen simultaneously. It is a way of seeing the world as a
51 system of pure statements and acting to make those statements
52 true. It is both ontology and epistemology.

53 First, therefore, I must establish that computation is the
54 apex of a long human search for a language of truth. The ori-
55 gin of symbol itself in the σύμβολον, a token broken in half to
56 infallibly mark identity, is one font. Later "symbol" was the
57 profession of faith, both an attempt to formulate truth and a
58 sign of shared claims. The primary source, however, is lan-
59 guage itself, Hofstadter's "strange loop" of self-reference. The
60 first possible Nock formula is a crash, the void. The second is
61 self-reference, awakening. Peirce would be proud of the pro-
62 gression. We have tricked ourselves into thinking that com-
63 puting is a late development, a fruitful elaboration of the end
64 of mathematics. In fact, it is the philosopher's stone, and it
65 underlies every process in the world. It has turned lead into
66 gold.

67 The historical account which follows aims throughout at
68 the goal of understanding what Nock is and why it matters as
69 a protocol. The reader will understand what mankind has been
70 looking for, when we have caught various pieces of the puzzle,
71 how these things came together in the mid-20th century, and
72 how they have been refined into a pure machine.

73 Outline: - Cuneiform, tabulation, calculation, and algorithm
74 - Solid-state indices: abaci, tokens, etc. Mechanism as em-
75 bodied equation. - Pure language and true statements (Lull,
76 Leibniz, Wilkins) - Generalization (Jacquard, Babbage) - The
77 universal machine (Turing, Gödel, Zuse, von Neumann) - Pure
78 statements as Pages from the Book (Curry, Church)

79 **1.1 Tabulation**

80 **1.2 Solid-state indices**

81 The problem is that any calculation lives in someone's head
82 or someone's claim. This is a summary, but not the proof or
83 solution.

84 Chapter 2

85 Combinators for Fun 86 and Profit

87 Abstract

88 The first section of this chapter will introduce the basic SKI
89 and BCKW combinator calculi and examine how they can rep-
90 resent each other. The second section will introduce Nock’s
91 opcodes as combinators, and show how they can be used to ex-
92 press various combinator idioms. The third section will show
93 how to build practical affordances for common programming
94 tasks, such as conditionals, recursion, subprocedure invoca-
95 tion, and data structures. The final section will discuss the re-
96 lationship between combinators and virtualization, and how
97 Nock can be used to build a virtual machine even in a crash-
98 only context.

99 2.1 Combinators in Computing

100 The pioneers of computing elaborated three complementary vi-
101 sions:

- 102 1. The Turing machine, a simple abstract machine that can
103 compute anything computable.

2. The lambda calculus, a pure functional language based on substitution and abstraction.
3. Combinatory logic, a variable-free functional language based on combinators.

Logicians early demonstrated the equivalence of these approaches, but particular formulations are still preferred for particular tasks. The Turing machine is the most intuitive model of computation, and is often used to prove computability results. The lambda calculus is the basis of many functional programming languages, such as Lisp, Scheme, and Haskell. Combinatory logic is less widely used in practice, but has a long history in mathematical logic and type theory. It is also the basis of Nock.

Before we get too far out over our skis, let's talk about what a combinator is. At its simplest, a combinator is a function that takes one or more arguments and returns a result, without referring to any variables outside its own arguments. In practice, we'll use different combinators to build up more complex functions, and we'll see informally how they can be combined to express any computable function. We can even build combinators out of other combinators. The objective of combinatory logic is to express all computation in terms of combinators alone, without the need for variables or named functions.

This first section will introduce some elemental and compound combinators and make the case that they circumscribe "computing" as a concept.

2.1.1 The SKI calculus

Computable functions can be expressed using minimal sets of combinators. Many logicians today prefer to work with Moses Schönfinkel and Haskell Curry's SKI combinator calculus, which has three combinators:

The identity combinator I is conventionally written $Ix = x$. This simply means that it returns its single argument unchanged. There is little to say of this combinator at this point, but logicians have found that it frequently simplifies expressions.

140 $I\ x$

141 x

142 The constant combinator K is conventionally written Kxy
 143 $= x$. This means that it takes two arguments and returns the
 144 first, ignoring the second. It is useful for building functions
 145 that always return a fixed value.

146 Questions

147 1. What is the result of $I\ x$?

148 2. What is the result of $I\ I\ x$?

149 $K\ x\ y$

150 x

151 The substitution combinator S is conventionally written $Sxyz$
 152 $= xz(yz)$. This means that it takes three arguments: a function
 153 x , and two values y and z . It applies the function x to the value
 154 z , and also applies the function y to the value z , then applies
 155 the result of $x\ z$ to the result of $y\ z$. This combinator is more
 156 complex than the others, but it is very powerful, as it allows us
 157 to express function application and composition.

158 $S\ x\ y\ z$

159 $x\ z\ (y\ z)$

160 The SKI system is Turing complete (**blah, blah**), mean-
 161 ing that any computable function can be expressed using only
 162 these three combinators. It is also equivalent to the lambda cal-
 163 culus, meaning that any lambda expression can be translated
 164 into an equivalent SKI expression and vice versa. (cite church-
 165 turing thesis stuff)

166 Combinators can be combined to form more complex ex-
 167 pressions. For example, the combinator $S\ K\ K$ is equivalent to
 168 the identity combinator I :

169 $S\ K\ K\ x$

170 $K\ x\ (K\ x)$

171 x

2.1.2 The BCKW calculus

The BCKW combinator calculus is an alternative to SKI, with a different set of primitive combinators. It has four combinators:

The composition combinator B is conventionally written $Bxyz = x(yz)$. This means that it takes three arguments: a function x , and two values y and z . It applies the function y to the value z , then applies the function x to the result.

```
B x y z
x (y z)
```

The SKI and BCKW systems are equivalent, meaning that any expression in one system can be translated into an equivalent expression in the other system. For example, the identity combinator I can be expressed in BCKW as:

```
I = W K
    U
    Y
Y = S (K (S I I)) (S (S (K S) K) (K (S I I)))
```

WABT or whatever it is

Chris Barker The iota combinator ι can produce all the others, but in convoluted ways.

Combinator calculi are generally considered too minimalist for practical programming; they shine, however, in mathematical proofs. This means that we can use them to define a standard of behavior for an algorithm, but implement the logic in a compliant form using whatever computational tools we like. Thus it is better to think of Nock not as a bare programming language, but as a standard of computation which can under many circumstances be used as the computation itself.¹

2.2 Nock Opcodes as Combinator

The Nock specification is produced in full in Listing 2.1.

¹Consideration of when to do so is largely driven by performance bottlenecks.

Listing 2.1: Nock 4K

```

202
203 Nock 4K
204
205 A noun is an atom or a cell. An atom is a natural
206     number. A cell is an ordered pair of nouns.
207
208 Reduce by the first matching pattern; variables match
209     any noun.
210
211 nock(a)                *a
212 [a b c]                [a [b c]]
213
214 ?[a b]                 0
215 ?a                     1
216 +[a b]                 +[a b]
217 +a                     1 + a
218 =[a a]                 0
219 =[a b]                 1
220
221 /[1 a]                 a
222 /[2 a b]               a
223 /[3 a b]               b
224 /[(a + a) b]           /[2 /[a b]]
225 /[(a + a + 1) b]       /[3 /[a b]]
226 /a                     /a
227
228 #[1 a b]               a
229 #[(a + a) b c]         #[a [b /[(a + a + 1) c]] c]
230 #[(a + a + 1) b c]     #[a [/[(a + a) c] b] c]
231 #a                     #a
232
233 *[a [b c] d]           [*[a b c] *[a d]]
234
235 *[a 0 b]               /[b a]
236 *[a 1 b]               b
237 *[a 2 b c]             *[*[a b] *[a c]]
238 *[a 3 b]               ?*[a b]
239 *[a 4 b]               ++[a b]
240 *[a 5 b c]             =[*[a b] *[a c]]
241
242 *[a 6 b c d]           *[a *[[c d] 0 *[[2 3] 0 *[a 4 4
243     b]]]]

```

244	<code>*[a 7 b c]</code>	<code>*[*[a b] c]</code>
245	<code>*[a 8 b c]</code>	<code>*[[*[a b] a] c]</code>
246	<code>*[a 9 b c]</code>	<code>*[*[a c] 2 [0 1] 0 b]</code>
247	<code>*[a 10 [b c] d]</code>	<code>#[b *[a c] *[a d]]</code>
248		
249	<code>*[a 11 [b c] d]</code>	<code>*[[*[a c] *[a d]] 0 3]</code>
250	<code>*[a 11 b c]</code>	<code>*[a c]</code>
251		
252	<code>*a</code>	<code>*a</code>
253		

2.2.1 I Identity = Opcode 0

Nock's binary tree addressing is fully general and yields the identity combinator I as a special case. (The cost, of course, is dragging in some machinery for basic arithmetic, as we will see.)

Conceptually, we could write this in a hybrid form between combinators and Nock thus:

```
*[a I b] == [0 1]
```

2.2.2 K Constant = Opcode 1

2.2.3 S Substitution = Opcode 2

2.2.4 The Axiomatic Operators

Because of Nock's cellular nature as a binary tree, we need the ability to access and manipulate nouns.

opcode 3, 4, 5, 10

2.3 Idioms & Design Patterns

Several of the combinators we examined above are not present in Nock *simpliciter*, but we can build them out of the other pieces. There are also common idioms which are baked in as "compound opcodes" (such as conditional branching, opcode 6).

2.3.1 C Reversal

$C = S (S (K (S (K S) K)) S) (K K)$

In Nock, the reversal combinator C is almost free due to the binary tree nature of cells.

```
*[a C b] == *[7 [0 3] [[0 3] [0 2]]]
*[a C b] == *[[0 7] [0 6]]
```

Recursion we get for free with opcode o .

μ -recursive functions

2.4 Virtualization

Nock is a crash-only pure language, which means at the base level it can only succeed and yield a new noun: no side effects, no exceptions, no state. Like a castaway on a desert island, we must build everything we need from raw materials at hand.

Nock has one trick built in and a couple of design patterns to help us get there:

1. Opcode 11 lets us send a hint to the runtime, which can be used to trigger side effects. The Nock expression formally doesn't know anything about these, but we can use them to print results, read input, and so forth.
2. The runtime can maintain state outside of Nock and supply it as a subject to subsequent Nock expressions. We will do a lot more with this later, but we want to highlight it as a key affordance now.
3. The virtualization pattern means that we can run Nock in Nock.

Opcode 11 can encode anything we want: the result of a hint is used by the runtime, and so anything the runtime is willing to work with can be produced.

The magic formula at the beating heart of a practical Nock operating system is simply:

```
*([state -.events] [9 2 10 [6 0 3] 0 2])
```

311 That is, apply the Nock formula against a subject pair consist-
312 ing of the current state and the head of the list of events to
313 process.

314 Chapter 3

315 Interpreter Runtime

316 Abstract

317 Now that we have established a firm foundation for id-
318 iomatic programming in Nock, we can build an interpreter us-
319 ing a higher-level language to execute Nock formulas. At this
320 point, we will build the simplest possible Nock interpreter, a
321 tree-walking interpreter. This requires a cursor and some basic
322 memory management but not much more. We will implement
323 some minimalist static hints to permit side effects like output.

